



《系统软件原理与设计III-数据库系统》

第5章规范化理论

计算机科学与工程学院（网络空间安全学院）

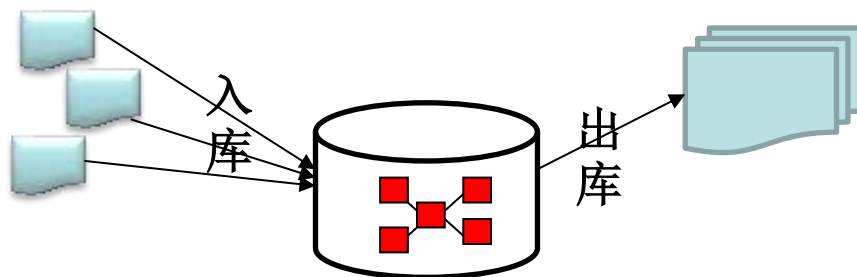
任课教师：胡旺

2025年12月14日星期日



➤ 存取数据存在哪些问题？

- 如何组织这些数据？—— 数据模型、规范化理论、设计方法
- 如何存取这些数据？—— 数据定义和操作语言
- 哪些人可以操作哪些数据？—— 安全性控制
- 多人如何操作同一数据？—— 并发性控制
- 出现故障后怎么办？—— 数据恢复
- 如何分析数据和发现数据价值？—— 数据仓库、数据挖掘





本章内容



1

关系模式设计问题

2

函数依赖

3

模式分解

4

规范化

5

*多值依赖

6

*连接依赖



数据库设计问题



■ 如何构造一个合适的数据库模式？

- ◆ 应该构造几个关系模式？
- ◆ 每个关系应该由哪些属性组成？

■ 思考：教务管理数据库模式？

■ 数据库逻辑设计的工具

- ◆ 关系数据库的规范化理论



- 关系模式的五元组表示： $R(U, D, DOM, F)$
 - ◆ R: 关系名
 - ◆ U: 组成该关系的属性名集合
 - ◆ D: 属性组U中属性所来自的域
 - ◆ DOM: 属性向域的映象集合
 - ◆ F: 属性间数据的依赖关系集合
- 关系模式的简化三元组表示： $R(U, F)$
- 数据依赖举例
 - ◆ 医生姓名、医生职称、医生年龄与医生编号之间的关系？



■ 关系模式实例

- ◆ 设有一个医生与患者之间的就诊关系模式R (Dname, Dlevel, Dsal, Pname, Fsum), 其属性分别表示医生姓名、医生职称级别、医生工资、患者姓名、诊治费用。

■ R上的语义:

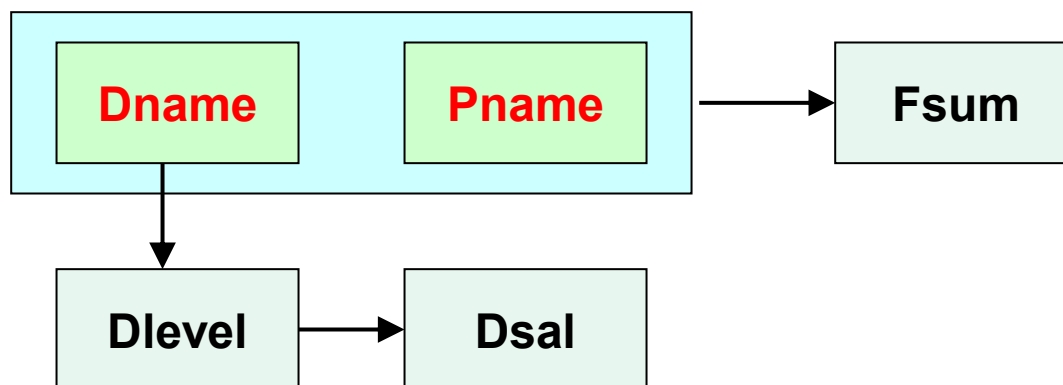
- ◆ 假设医生和患者的姓名分别都是唯一的。
- ◆ 医生与患者之间是多对多的关系, 即医生可以为不同的患者看病, 同时患者可以选择不同的医生。假设同一名患者不看相同的医生, 即可以选择Dname和Pname作为就诊关系模式R的主键。
- ◆ 一位患者每次就诊都有一个花销总金额。
- ◆ 每位医生具有相应的职称级别。
- ◆ 职称级别决定了医生的工资金额。



Dname	Dlevel	Dsalary	Pname	Fsum
罗晓	主任医师	3200	张珍	¥30.00
杨勋	副主任医师	2800	张珍	¥50.00
杨勋	副主任医师	2800	刘景	¥55.00
杨勋	副主任医师	2800	张柳	¥58.00
邓英超	主治医师	2400	李秀	¥75.00
罗晓	主任医师	3200	傅伟相	¥35.00



■ 可以确定以下函数依赖：

$$F = \{ \{Dname, Pname\} \rightarrow Fsum, \\ Dname \rightarrow Dlevel, \\ Dlevel \rightarrow Dsal \}$$




■ 就诊模式R存在的问题？（虽然只有5个属性）

◆ 数据冗余：浪费存储空间，引起异常。

◆ 操作异常：

- 更新异常（Update Anomalies）。
- 删除异常（Delete Anomalies）。
- 插入异常（Insert Anomalies）。

■ 因此，就诊关系模式R**不是**一个好的关系模式。

Dname	Dlevel	Dsalary	Pname	Fsum
罗晓	主任医师	3200	张珍	¥30.00
杨勋	副主任医师	2800	张珍	¥50.00
杨勋	副主任医师	2800	刘景	¥55.00
杨勋	副主任医师	2800	张柳	¥58.00
邓英超	主治医师	2400	李秀	¥75.00
罗晓	主任医师	3200	傅伟相	¥35.00



■ 消除冗余和异常现象 (模式分解)

- ◆ R1 (Dname, Dlevel)
- ◆ R2 (Dlevel, Dsal)
- ◆ R3 (Dname, Pname, Fsum)

Dname	Dlevel	Dsalary	Pname	Fsum
罗晓	主任医师	3200	张珍	¥30.00
杨勋	副主任医师	2800	张珍	¥50.00
杨勋	副主任医师	2800	刘景	¥55.00
杨勋	副主任医师	2800	张柳	¥58.00
邓英超	主治医师	2400	李秀	¥75.00
罗晓	主任医师	3200	傅伟相	¥35.00

(a) R1 的实例

Dname	Dlevel
罗晓	主任医师
杨勋	副主任医师
邓英超	主治医师

(b) R2 的实例

Dlevel	Dsalary
副主任医师	2800
主任医师	3200
主治医师	2400

(c) R3 的实例

Dname	Pname	Fsum
罗晓	张珍	¥30.00
杨勋	张珍	¥50.00
杨勋	刘景	¥55.00
杨勋	张柳	¥58.00
邓英超	李秀	¥75.00
罗晓	傅伟相	¥35.00



本章内容



1

关系模式设计问题



2

函数依赖

3

模式分解

4

规范化

5

*多值依赖

6

*连接依赖



■ 什么是数据依赖？

- ◆ 是现实世界属性间相互联系的抽象
- ◆ 是数据内在的性质
- ◆ 是语义的体现

■ 数据依赖的类型

- ◆ 函数依赖 (Functional Dependency, 简记为FD)
- ◆ 多值依赖 (Multivalued Dependency, 简记为MVD)



- 函数依赖是指一个关系表中**属性（列）之间的联系**。
- 函数依赖是关系中属性之间在语义上的关联特性。
- 函数依赖关注一个属性或属性集与另外一个属性或属性集之间的依赖，亦即两个属性或属性集之间的约束。
- 数据库**设计者**根据对关系R中的属性的**语义**理解确定函数依赖，确定约束R的所有元组r的函数依赖集，并获知属性间的语义关联。



■ 符号说明:

- ◆ R 表示一个关系的模式;
- ◆ $U = \{A_1, A_2, \dots, A_n\}$ 是 R 的所有属性的集合;
- ◆ F 是 R 中函数依赖的集合;
- ◆ r 是 R 所取的值;
- ◆ $t[X]$ 表示元组 t 在属性 X 上的取值。例如 $t[Dname] = \text{'杨勋'}$

■ 函数依赖定义

设有一关系模式 $R(U)$, X 和 Y 为其属性 U 的子集, 即 $X \subseteq U$, $Y \subseteq U$ 。设 t, s 是关系 R 中的任意两个元组, 如果 $t[X] = s[X]$, 则 $t[Y] = s[Y]$ 。那么称 Y 函数依赖于 X , 或 X 函数决定 Y 。也可称 FD $X \rightarrow Y$ 在关系模式 $R(U)$ 上成立。

■ 函数依赖图

- ◆ 左部称为决定因子
- ◆ 右部称为依赖因子。





■ 定义

如果 $Y \subseteq X$ ，显然 $X \rightarrow Y$ 成立，这称为平凡函数依赖 (Trivial Functional Dependency)。

- 平凡函数依赖必然成立，它不反映新的语义。

例如： $\{Dname, Pname\} \rightarrow \{Pname\}$ 。

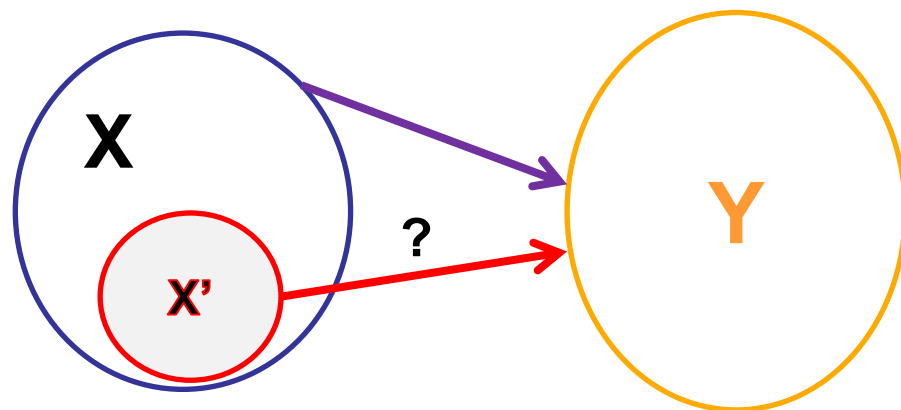
- 平常所指的函数依赖一般都指非平凡函数依赖。



■ 定义

设 X 、 Y 是某关系的不同属性集，如 $X \rightarrow Y$ ，且不存在 $X' \subset X$ ，使 $X' \rightarrow Y$ ，则 Y 称完全函数依赖 (Full Function Dependency) 于 X ，记为 $X \xrightarrow{f} Y$ ；否则称 Y 部分依赖 (Partial Functional Dependency) 于 X ，记为 $X \xrightarrow{p} Y$ 。

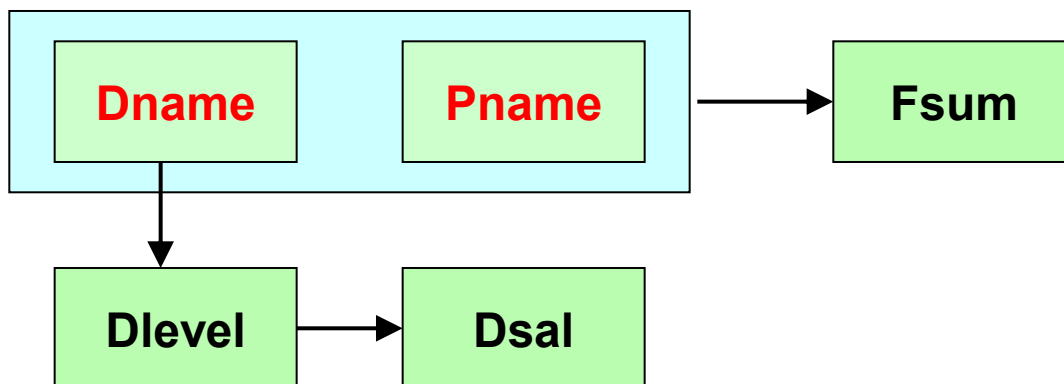
- 完全函数依赖用来表明函数依赖的决定因子中的**最小属性集**。
- 属性集 Y 完全函数依赖于属性集 X ，如果满足下列条件：
 - ◆ Y 函数依赖于 X 。
 - ◆ Y 不函数依赖于 X 的任何真子集。





■ 举例

在 $R(Dname, Dlevel, Dsal, Pname, Fsum)$ 中, $\{Dname, Pname\}$ 是主键, 故 $\{Dname, Pname\} \rightarrow Dlevel$, 但 $Dname \rightarrow Dlevel$, 故 $\{Dname, Pname\} \xrightarrow{p} Dlevel$, 而 $\{Dname, Pname\} \xrightarrow{f} Fsum$ 。





■ 定义

设 X, Y, Z 是某关系的不同的属性集, 如果 $X \rightarrow Y, Y \twoheadrightarrow X, Y \rightarrow Z$, 则称 Z 对 X 传递函数依赖 (Transitive Functional Dependency)。

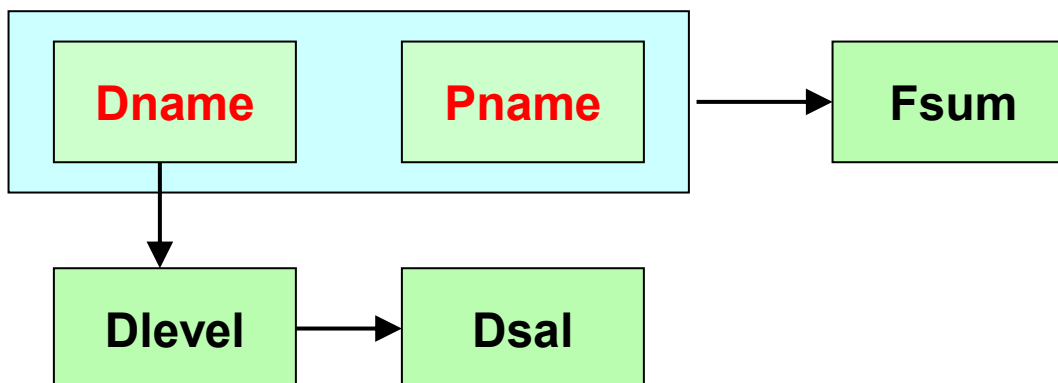
■ 直接依赖

由于有了条件 $Y \twoheadrightarrow X$, 说明 X 与 Y 不是一一对应的; 否则, $X \leftrightarrow Y, Z$ 就直接函数依赖于 X , 而不是传递函数依赖于 X 了。



■ 举例：

在就诊关系R中，存在函数依赖 $Dname \rightarrow Dlevel$ ， $Dlevel \rightarrow Dsal$ ，所以 $Dname \rightarrow Dsal$ 。





- 函数依赖中需要解决的问题：从一些已知的函数依赖去判断另外一些函数依赖是否成立。（逻辑推理）
- 例如，如果 $A \rightarrow B$ 和 $B \rightarrow C$ 在某个关系中成立，记： $F = \{ A \rightarrow B, B \rightarrow C \}$ ，那么 $A \rightarrow C$ 在该关系中是否成立的问题称为**逻辑蕴涵问题**，若 $A \rightarrow C$ 成立，则说 F 逻辑蕴涵 $A \rightarrow C$ ，记作： $F \models A \rightarrow C$ 。
- 定义

设 F 是 R 的函数依赖集， X, Y 是 R 的属性子集， $X \rightarrow Y$ 是 R 的一个函数依赖。如果一个关系模式 R 满足 F ，则必然满足 $X \rightarrow Y$ ，这时称 F 逻辑蕴涵 $X \rightarrow Y$ ，或表示为 $F \models X \rightarrow Y$ 。



- 一个关系模式可能有多个函数依赖形成函数依赖集，现在有一个新的函数依赖不在函数依赖集里，但能从集合里根据一定的**规则**推导出来，就说那个集合**逻辑蕴涵**（隐含）这个新的函数依赖。

■ 举例

$X \rightarrow Z$ 并不是显式地表现出来，而是从 $X \rightarrow Y$ 和 $Y \rightarrow Z$ 推出的，这可表示为 $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$

- 如果一个函数依赖能够由集合中的其他函数推出，则该函数依赖是多余的。



■ 闭包(*)

函数依赖集合 F 所逻辑蕴涵的函数依赖的全体称为 F 的闭包 (Closure), 记为 F^+ , 即 $F^+ = \{X \rightarrow Y \mid F \models X \rightarrow Y\}$ 。

- 函数依赖集的闭包（也成为完备集）定义了由给定函数依赖集所能推导出的所有函数依赖。
- 通过 F 得到 F^+ 的算法可以由 Armstrong 公理推导出来。



- **无冗余**的函数依赖集和函数依赖的**完备集（闭包）**是好的关系设计。
- 根据已知函数依赖集，推导其它函数依赖时所依据的规则称为**推理规则**。
- 函数依赖的推理规则最早出现在Armstrong的论文里。这些规则常被称为“Armstrong公理”。



- 设 U 是关系模式 R 的属性集, F 是 R 上成立的只涉及 U 中属性的函数依赖集。FD推理规则:

- (1) 自反性 (Reflexivity)

如果 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 成立。这是一个平凡函数依赖。

- (2) 增广性 (Augmentation)

如果 $X \rightarrow Y$ 成立, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ 成立。

式中, XZ 和 YZ 是 $X \cup Z$ 和 $Y \cup Z$ 的简写。

- (3) 传递性 (Transitivity)

如果 $X \rightarrow Y$, $Y \rightarrow Z$ 成立, 则 $X \rightarrow Z$ 成立。



■ (*扩展规则)

合并性 (Union): $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$

伪传递性 (Pseudo-transitivity): $\{X \rightarrow Y, WY \rightarrow Z\} \models XW \rightarrow Z$

分解性 (Decomposition): $\{X \rightarrow Y, Z \subseteq Y\} \models X \rightarrow Z$

复合性 (Composition): $\{X \rightarrow Y, W \rightarrow Z\} \models XW \rightarrow YZ$

通用一致性 (General Unification): $\{X \rightarrow Y, W \rightarrow Z\} \models X \cup (W - Z) \rightarrow YZ$



- Armstrong公理是**正确的 (Sound)**，即如果 $X \rightarrow Y$ 是从 F 用推理规则导出，那么 $X \rightarrow Y$ 在 F^+ 中。
- Armstrong公理 (FD推理规则 $\{A1, A2, A3\}$) 是**完备的 (Complete)**。
- 推理规则的正确性是指“从FD集 F 使用推理规则推出的FD**必定在 F^+ 中**”，而推理规则的完备性是指“ **F^+ 中的FD都能从 F 集使用推理规则推出**”。
- 即正确性保证了推出的所有FD都是正确的，完备性保证了可以推出所有被蕴涵的FD。这些就保证了推导的有效性和可靠性。



■ 候选码与主码：用函数依赖定义

设 K 为 $R(U, F)$ 中的属性或属性组合，若 $K \xrightarrow{f} U$ ，则 K 为 R 的候选码 (Candidate Key)。若候选码多于一个，则选定其中的一个为主码 (Primary Key)。

- 包含在任何候选码中的属性称为**主属性 (Prime Attribute)**。不包含在任何候选码中的属性称为**非主属性 (Non-Key Attribute)**。
- 最简单的情况，单个属性是码。最极端的情况，整个属性组是码，称为**全码 (All-key)**。



■ 候选码与主码选择

- ◆ 如果学生关系模式中有 **SID**（自增长整数或128位随机字符）、**学号**、**身份证号**等**候选码属性**，你认为选择哪个作为主键更合适？





■ 候选码与主码的举例

- ◆ 在关系模式 R_1 (Dname, Dlevel, Dsal) 中 Dname 是码
- ◆ 在关系模式 R_2 (Dname, Pname, Fsum) 中的属性组合 (Dname, Pname) 是码。
- ◆ 关系模式 R_3 (Dname, Pname, Mname) 表示医生给患者开具的药品，假设一个医生可以给多个患者看病，一位患者可以选择不同的医生就诊，不同的医生可以给患者开具不同的药品，因此 (Dname, Pname, Mname) 为 R_3 的码，即全码。



■ 外码：用函数依赖定义

关系模式 R_1 中属性或属性组合 X 并非 R_1 的码，但 X 是另一个关系模式 R_2 的码，则称 X 是 R_1 的外码 (Foreign Key)。

■ 举例：

- ◆ 在关系模式 R_2 (Dname, Pname, Fsum) 中，Dname不是码，
- ◆ 但Dname是关系模式 R_1 (Dname, Dlevel, Dsal) 的码，
- ◆ 则Dname是关系模式 R_2 的外码。



■ 主码与外码提供了一个表示关系之间联系的手段。如上述关系模式 R_1 和 R_2 的联系就是通过Dname来体现的。



由 FD 推理规则的分解性可知, 如果 $X \rightarrow \{A_1, A_2, \dots, A_k\}$, 则 $X \rightarrow A_i (i=1, 2, \dots, k)$ 。
由合并性可知, 如果 $X \rightarrow A_i (i=1, 2, \dots, k)$, 则 $X \rightarrow \{A_1, A_2, \dots, A_k\}$,
故 $X \rightarrow \{A_1, A_2, \dots, A_k\}$ 和 $X \rightarrow A_i (i=1, 2, \dots, k)$ 是等价的。
由此可知, 候选码唯一地决定一个元组; 也可以说, 候选码决定每个属性。

医生编号	医生姓名	医生性别	医生年龄	所属部门	技术等级	工资
------	------	------	------	------	------	----

主键 (Primary Key, PK)



- 判断能否从已知的FD集推导出FD $X \rightarrow Y$, 那么可先求出 F 的闭包 F^+ , 然后再看 $X \rightarrow Y$ 是否在 F^+ 中。
- 但是从 F 求 F^+ 是一个**NP完全问题** (指数级的时间复杂度)。引入属性集闭包概念, 将使该问题化为**多项式级**的时间复杂度问题。
- 属性集闭包的定义

设 $X \subset U$, F 是属性集 U 上的 FD 集, 那么相对于 F , 属性集 X 的闭包用 X^+ 表示, 它是一个从 F 集使用 FD 推理规则推出的所有满足 $X \rightarrow A$ 的属性 A 的集合:

$$X^+ = \{A \mid A \in U \text{ 且 } X \rightarrow A \text{ 在 } F^+ \text{ 中}\}$$

求属性集 U 为 $ABCD$, $FD = \{A \rightarrow B, B \rightarrow C, D \rightarrow B\}$ 。

用上述算法可求出 $A^+ = ABC$, $(AD)^+ = ABCD$, $(BD)^+ = BCD$ 。



■ 定理

$X \rightarrow Y$ 能用 FD 推理规则导出的充分必要条件是 $Y \subseteq X^+$ 。

- 从属性集 X 求 X^+ 并不太难，花费的时间与 F 中的 FD 数目成正比，是一个多项式级时间复杂度的问题。

■ 求属性集闭包的算法

求属性集 X 相对于 FD 集 F 的闭包 X^+ 。

设属性集 X 的闭包为 closure ，其计算算法如下：

```
closure = x;  
do {  
    if    F 中有某个 FD  $U \rightarrow V$  满足  $U \subseteq \text{closure}$   
    then  $\text{closure} = \text{closure} \cup V$ ;  
}  
while( closure 有所改变 );
```



■ 函数依赖集等价: $F = G$

设 F, G 为关系模式 $R(U)$ 上的两个函数依赖集, 如果 $F^+ = G^+$, 则称 F 和 G 是等价的函数依赖集, 也可称 F 覆盖 G , 或 G 覆盖 F , 也可说 F, G 互相覆盖。

■ 设 F 是属性集 U 上的函数依赖集, 如果 $F_{\min}$ 是 F 的一个最小依赖集, 那么 $F_{\min}$ 应满足下列4个条件:

- ① $F_{\min}^+ = F^+$ 。
- ② F 中每个函数依赖的右部为单属性。
- ③ $F_{\min}$ 中没有冗余的 FD, (即在 $F_{\min}$ 中不存在这样的函数依赖 $X \rightarrow Y$, 使得 $F_{\min} - \{X \rightarrow A\}$ 与 $F_{\min}$ 等价)。
- ④ 每个 FD 的左边没有冗余的属性 (即在 $F_{\min}$ 中也不存在这样的 $X \rightarrow Y$, 使得 $F_{\min} - \{X \rightarrow Y\} \cup \{W \rightarrow Y\}$ 与 $F_{\min}$ 等价, 式中, $W \subset X$)。



■ 求最小函数依赖集的算法

计算函数依赖集 F 的最小依赖集 G 。

第一步：根据推理规则的分解性，得到一个与 F 等价的 FD 集 G ， G 中每个 FD 的右边均为单属性。

第二步：在 G 的每个 FD 中消除左边冗余的属性。

第三步：在 G 中消除冗余的 FD。

■ 举例

设 F 是关系模式 $R(ABC)$ 的 FD 集， $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$ ，试求 $F_{\min}$ 。

解：① 先将 F 中的 FD 写成右边为单属性形式：

$$F = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$$

删除上式中重复的 $A \rightarrow B$ ，得 $F = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C\}$ 。

② F 中的 $A \rightarrow C$ 可以由 $A \rightarrow B$ 和 $B \rightarrow C$ 推出，因此可以删除冗余的 $A \rightarrow C$ ，得 $F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow C\}$ 。

③ F 中 $AB \rightarrow C$ 可以从 $B \rightarrow C$ 推出，因此可以删除冗余的 $AB \rightarrow C$ ，最后得到 $F = \{A \rightarrow B, B \rightarrow C\}$ ，即所求的 $F_{\min}$ 。

■ 每个FD至少存在一个 $F_{\min}$ ，但不一定唯一。



本章内容



1

关系模式设计问题

2

函数依赖



3

模式分解

4

规范化

5

*多值依赖

6

*连接依赖



- 由函数依赖可以引起的**更新异常问题**，同样，多值依赖和连接依赖也会引起类似的问题。
- 解决这些问题的途径：
 - ◆ 按照“**一事一地**”的原则设计模式，使之表达的**语义概念单纯化**。

(a) R1 的实例		(b) R2 的实例		(c) R3 的实例		
Dname	Dlevel	Dlevel	Dsalary	Dname	Pname	Fsum
罗晓	主任医师	副主任医师	2800	罗晓	张珍	¥30.00
杨勋	副主任医师	主任医师	3200	杨勋	张珍	¥50.00
邓英超	主治医师	主治医师	2400	杨勋	刘景	¥55.00
				杨勋	张柳	¥58.00
				邓英超	李秀	¥75.00
				罗晓	傅伟超	¥35.00

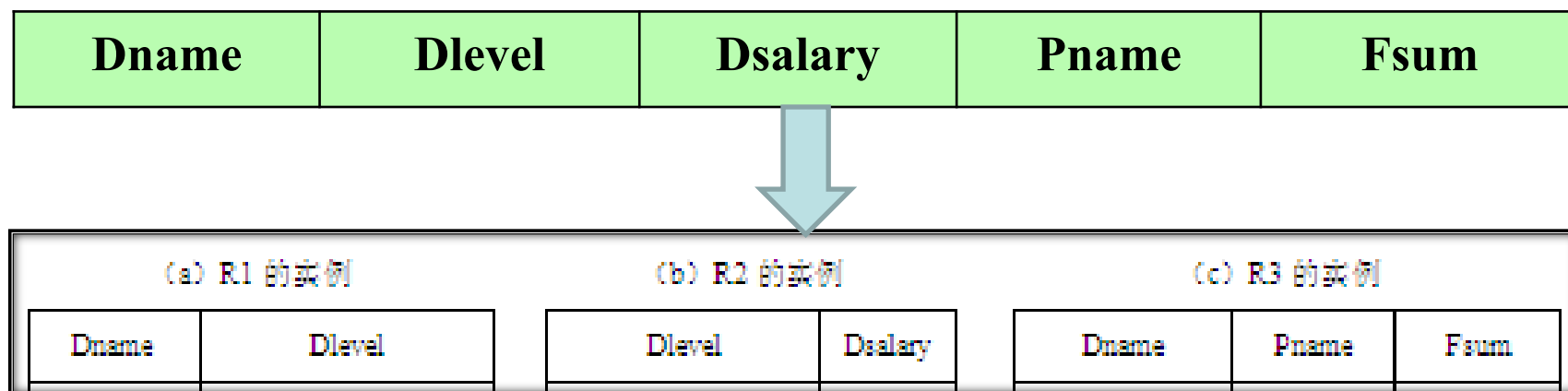
- ◆ 对“**不单纯**”的关系模式R进行分解：用多个关系来替换R，分解后的关系模式的属性集都是R中属性的子集，其并集与R的属性集相同。
- 模式分解帮助消除不良设计中的一些问题，如**冗余、不一致或异常**。



■ 模式分解的定义

设有关系模式 $R(U)$, 属性集为 U , 若用一关系模式的集合 $\{R_1(U_1), R_2(U_2), \dots, R_k(U_k)\}$ 来取代,

其中 $U = \bigcup_{i=1}^k U_i$, 则称此关系模式的集合为 R 的一个分解, 以 $\rho = \{R_1, \dots, R_k\}$ 表示。





■ 模式分解的问题

R 称为泛关系模式, R 对应的当前值 r 称为泛关系。数据库模式 ρ 对应的当前值 σ 称为数据库实例, 它是由数据库模式中的每一个关系模式的当前值组成, 用 $\sigma = \langle r_1, \dots, r_k \rangle$ 表示。但这里就有两个问题:

- σ 和 r 是否等价, 即是否表示同样的数据。这个问题用“无损分解”特性表示。
- 在模式 R 上有一个 FD 集 F , 在 ρ 的每一个模式 R_i 上有一个 FD 集 F_i , 那么 $\{F_1, \dots, F_k\}$ 与 F 是否等价。这个问题用“保持依赖”特性表示。



- 一个关系表被分解成两个或两个以上的小表，通过连接被分解后的小表可以获得原始表的内容，则称为**无损连接分解**。

设 R 是一个关系模式， F 是 R 上的一个 FD 集。 R 分解成数据库模式 $\rho = \{R_1, \dots, R_k\}$ 。如果对 R 中满足 F 的每一个关系 r ，有：

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$$

那么称分解 ρ 相对于 F 是“无损连接分解” (Lossless Join Decomposition)，简称为“无损分解”；否则，称为“损失分解” (Lossy Decomposition)。

- 例如：将 $R(X, Y, Z)$ 分解成 $R_1(X, Y)$ 和 $R_2(X, Z)$ ，如果 X 是 R_1 和 R_2 的共同属性或属性集，且存在函数依赖集 $F = \{X \rightarrow Y, X \rightarrow Z\}$ ，则该分解是无损的。



模式分解

无损分解



r	A	B	C
1	1	1	1
1	1	2	1

(a)

$r1$	A	B
1	1	1
1	1	2

(b)

$r2$	A	C
1	1	1

(c)

未丢失信息的分解

- ◆ 无损中的损是指**信息丢失**。如果一个分解不是无损分解，则所得结果的元组数总比原来的多（**增加了噪声，但把原来的信息丢失了**）。所谓“有损”就损在出现多余的元组上。

r	A	B	C
1	1	1	4
1	1	2	3

(a)

$r1$	A	B
1	1	1
1	1	2

(b)

$r2$	A	C
1	1	4
1	1	3

(c)

$r1 \bowtie r2$	A	B	C
1	1	1	4
1	1	1	3
1	1	2	4
1	1	2	3

(d)

丢失信息的分解



■ 无损分解测试算法——追逐法 (Chase) *

输入：关系模式 $R(A_1, \dots, A_n)$ ；

R 上成立的函数依赖集 F ；

R 上的分解 $\rho = \{R_1, \dots, R_k\}$ 。

输出： ρ 相对于 F 是否无损分解。

方法：

① 构造一张 k 行 n 列的矩阵表格，每列对应一个属性 A_j ，每行对应一个模式 R_i 。如果 A_j 在 R_i 中，那么在表格的第 i 行第 j 列处填上符号 a_j ；否则填上 b_{ij} 。

② 把表格看成模式 R 的一个关系，反复检查 F 中每个 FD 在表格中是否成立，若不成立，则修改表格中的值。修改方法如下。

对于 F 中一个 FD $X \rightarrow Y$ ，如果表格中有两行在 X 值上相等，在 Y 值上不相等，那么把这两行在 Y 值上也改成相等的值。如果 Y 值中有一个是 a_j ，那么另一个也改成 a_j ；如果没有 a_j ，那么用其中一个 b_{ij} 替换另一个值（尽量把下标 ij 改成较小的数）。一直到表格不能修改为止（这个过程称为 Chase 过程）。

③ 若修改的最后一张表格中有一行是全 a ，即 $a_1 a_2 \dots a_n$ ，那么称 ρ 相对于 F 是无损分解，否则称损失分解。



- Chase是一个普遍的算法，无论一个关系模式分解为多少个关系模式，都可以用此算法进行检验。
- 如果一个关系模式**一分为二**，可以简化检验。

设 $\rho = \{R_1(U_1), R_2(U_2)\}$ 是关系模式 $R(U)$ 的一个分解，
则 ρ 是无损分解的充分必要条件是：

$$(U_1 \cap U_2) \rightarrow (U_1 - U_2) \text{ 或 } (U_1 \cap U_2) \rightarrow (U_2 - U_1)。$$

■ 例如

医生关系模式 $R(Dno, Dlevel, Dsal)$ 上函数依赖集 $F = \{Dno \rightarrow Dlevel, Dlevel \rightarrow Dsal\}$ 。

若 $R_1(Dno, Dlevel)$ 和 $R_2(Dlevel, Dsal)$ 为 R 的一个模式分解，

因为 $Dlevel \rightarrow Dsal$ ，令 $U_1 = \{Dno, Dlevel\}$ ， $U_2 = \{Dlevel, Dsal\}$ 。

则 $U_1 \cap U_2 = \{Dlevel\}$ ， $U_2 - U_1 = \{Dsal\}$ 。

故 $(U_1 \cap U_2) \rightarrow (U_2 - U_1)$ 。

故 ρ 是无损分解。



- 所有的模式分解必须是无损的。
- 无损连接分解总是关于特定函数依赖集 F 定义的。
- 模式分解能消除数据冗余和操作异常现象。



■但是分解以后，检索操作需要做笛卡尔积或连接操作，这将付出时间代价。

■一般认为，为了消除冗余和异常现象，对模式进行分解是值得的。

■ 消除冗余和异常现象 (模式分解)

- ◆ R1 (Dname, Dlevel)
- ◆ R2 (Dlevel, Dsal)
- ◆ R3 (Dname, Pname, Fsum)

Dname	Dlevel	Dsalary	Pname	Fsum
罗晓	主任医师	3200	张珍	¥30.00
杨勋	副主任医师	2800	张珍	¥50.00
杨勋	副主任医师	2800	刘景	¥55.00
杨勋	副主任医师	2800	张柳	¥58.00
邓英超	主治医师	2400	李秀	¥75.00
罗晓	主任医师	3200	傅伟相	¥35.00

1



(a) R1 的实例

Dname	Dlevel
罗晓	主任医师
杨勋	副主任医师
邓英超	主治医师

(b) R2 的实例

Dlevel	Dsalary
副主任医师	2800
主任医师	3200
主治医师	2400

(c) R3 的实例

Dname	Pname	Fsum
罗晓	张珍	¥50.00
杨勋	张珍	¥50.00
杨勋	刘景	¥55.00
杨勋	张柳	¥58.00
邓英超	李秀	¥75.00
罗晓	傅伟相	¥35.00



- 模式分解的另一个特性是分解的过程中能否保持函数依赖集，如果不能保持函数依赖，那么数据的语义就会出现混乱。
- 模式分解要保持函数依赖，因为函数依赖集 F 中的每一个函数依赖都代表数据库的一个约束。
- 如果某个分解能保持函数依赖集，那么在数据输入或更新时，只要每个关系模式本身的函数依赖约束被满足，就可以确保整个数据库中的语义完整性不被破坏。显然这是一种良好的特性。



■ 定义

设 $\rho = \{R_1, \dots, R_k\}$ 是 R 的一个分解, F 是 R 上的 FD 集, 如果有 $\bigcup_{i=1}^k \Pi R_i(F) \models F$, 那么称分解 ρ 保持函数依赖集 F 。

■ 举例:

- ◆ 设医生关系模式 $R(Dname, Dlevel, Dsal)$ 。假设每个医生只有一个职称级别, 每个职称级别只有一个工资数目。那么 R 上函数依赖集 $F = \{Dname \rightarrow Dlevel, Dlevel \rightarrow Dsal\}$ 。
- ◆ 如果将 R 分解成 $\rho = \{R_1(Dname, Dlevel), R_2(Dname, Dsal)\}$, 可以验证这个分解是**无损分解**。
- ◆ R_1 上的 FD 是 $F_1 = \{Dname \rightarrow Dlevel\}$, R_2 上的 FD 是 $F_2 = \{Dname \rightarrow Dsal\}$ 。但是从这两个函数依赖推导不出在 R 上成立的函数依赖 $Dlevel \rightarrow Dsal$ 。因此分解 ρ 把函数依赖 $Dlevel \rightarrow Dsal$ 丢失了, 即 ρ **不保持函数依赖**。



- 关系模式分解的两个特性实际上涉及两个数据库模式的等价问题，这种等价包括**数据等价**和**依赖等价**两个方面。
- 数据等价是指两个数据库实例应表示同样的信息内容，用“**无损分解**”衡量。如果是无损分解，那么对关系反复的投影和连接都不会丢失信息。
- 依赖等价是指两个数据库模式应有相同的**依赖集闭包**。在依赖集闭包相等情况下，**数据的语义是不会出错的**。
- 违反数据等价或依赖等价的分解很难说是一个很好的设计模式。但是要同时达到无损分解和保持函数依赖的分解也不是一件容易的事情，需要认真对待。



■ 举例分析*

设关系模式 $R(ABC)$ ， $\rho = \{AB, AC\}$ 是 R 的一个分解。试分析分别在 $F_1 = \{A \rightarrow B\}$ ， $F_2 = \{A \rightarrow C, B \rightarrow C\}$ ， $F_3 = \{B \rightarrow A\}$ ， $F_4 = \{C \rightarrow B, B \rightarrow A\}$ 情况下， ρ 是否具有无损分解和保持 FD 的分解集。

- ① 相对于 $F_1 = \{A \rightarrow B\}$ ，分解 ρ 是无损分解且保持 FD 的分解。
- ② 相对于 $F_2 = \{A \rightarrow C, B \rightarrow C\}$ ，分解 ρ 是无损分解，但不保持 FD 集。因为 $B \rightarrow C$ 丢失了。
- ③ 相对于 $F_3 = \{B \rightarrow A\}$ ，分解 ρ 是损失分解但保持 FD 集的分解。
- ④ 相对于 $F_4 = \{C \rightarrow B, B \rightarrow A\}$ ，分解 ρ 是损失分解且不保持 FD 集的分解(丢失了 $C \rightarrow B$)。



本章内容



1

关系模式设计问题

2

函数依赖

3

模式分解



4

规范化

5

*多值依赖

6

*连接依赖



- **范式**（Normal Forma, NF）是一种关系的状态，也是衡量关系模式好坏的标准。
- 范式的种类（1NF, 2NF, 3NF, BCNF）与数据依赖有着直接的联系。在关系模式中存在函数依赖时就有可能存在数据冗余，引出数据操作异常现象。
 - ◆ 例如：就诊关系模式R（Dname, Dlevel, Dsal, Pname, Fsum）不是一个好的设计，因为存在**冗余信息（重复存储的职称和工资）**。数据冗余不仅浪费存储空间，而且会使数据库难以保持数据的一致性。
- 范式可以用于确保数据库模式中没有各种类型的异常和不一致性。为了确定一个特定关系是否符合范式要求，需要**检查关系中属性间的函数依赖，而不是检查关系中的当前实例。**



- 一个规范化的模式有**最小的数据冗余**，它要求除了与元组进行连接的外键（在另一个关系中是主键的属性组）之外，数据库实例中的其他属性的值都不能被复制（冗余）。

(a) R1 的实例

Dname	Dlevel
罗晓	主任医师
杨勋	副主任医师
邓英超	主治医师

(b) R2 的实例

Dlevel	Dealary
副主任医师	2800
主任医师	3200
主治医师	2400

(c) R3 的实例

Dname	Pname	Fsum
罗晓	张珍	¥30.00
杨勋	张珍	¥50.00
杨勋	刘景	¥55.00
杨勋	张柳	¥58.00
邓英超	李秀	¥75.00
罗晓	傅伟相	¥35.00

- 规范化是验证和改进逻辑数据库设计的工具，使得逻辑设计能够：
 - ◆ 满足特定约束
 - ◆ 避免不必要的数据重复。



- 定义：在关系模式R的每个关系r中，如果每个属性值都是**不可再分的原子值**，那么称R是满足第一范式（1NF）的模式。
- 1NF不允许每个元组的每个属性对应一组值、一行值或两个值的组合。即1NF中不允许“**表中有表**”的现象。
- 1NF是关系模式应具有的最起码的条件。
 - ◆ 满足1NF的关系称为规范化的关系；否则称为非规范化的关系。
 - ◆ 关系数据库研究的关系都是规范化的关系。

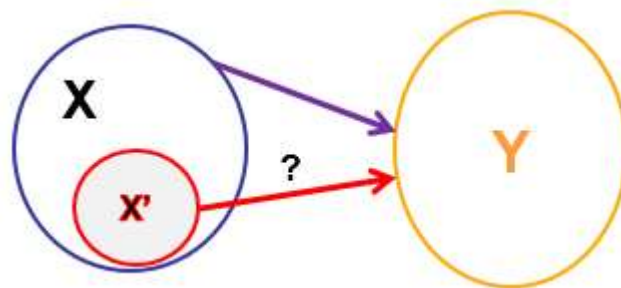
职工号	姓名	职称	工资			扣除		实发
			基本工资	工龄工资	职务工资	房租费	水电费	
88051	陈平	讲师	105	9.5	15	6	12	115.5
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.



- 例：就诊关系模式 R ($Dname, Pname, Dlevel, Dsal, Fsum$) 中每个属性都不可再分，因此 R 满足1NF。
- 但在医生关系模式 D ($Dname, Dresume$) 中，如果 $Dresume$ 包括**工作单位和工作时间两列信息**，某个医生可能曾经工作过多个单位而出现**多值**，则 D 不满足1NF。
 - ◆ 将 D 修改为 $D1$ ($Dname, Dorganise, Ddate$)，则 $D1$ 就满足了1NF。



- 如果关系模式中存在**部分函数依赖**（？），那么它就不是一个好的关系模式，因为它很可能出现**数据冗余**和**操作异常**现象。
 - ◆ 需要对这样的关系模式进行分解，以排除部分函数依赖，使模式达到2NF的标准。

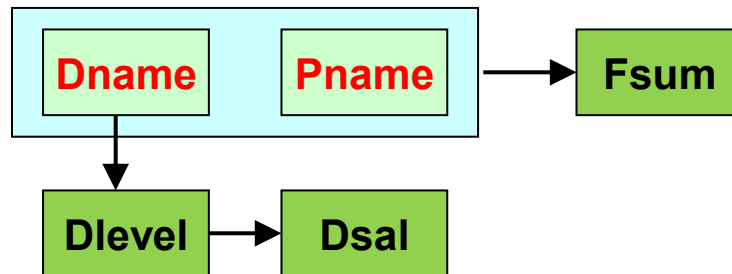


- 2NF定义： 如果关系模式 $R \in 1NF$ ，且每个**非主属性**（不是组成候选码的属性）**完全函数依赖**于候选码，那么称R属于2NF的模式。
- 2NF是基于完全函数依赖的，只有在主键是**复合属性**下才可能不符合2NF。
- 2NF是通往更高范式的中间步骤，它消除了1NF存在的部分问题。



■ 2NF举例

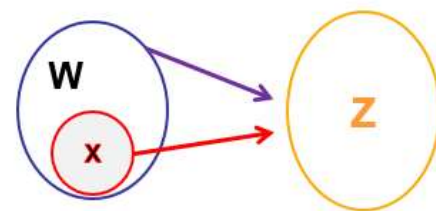
- ◆ 关系模式 R ($Dname$, $Pname$, $Dlevel$, $Dsal$, $Fsum$), 其中 ($Dname$, $Pname$) 是 R 的候选码。
- ◆ 如果 R 上有两个FD: $(Dname, Pname) \rightarrow (Dlevel, Dsal)$ 和 $Dname \rightarrow (Dlevel, Dsal)$, 则前面一个FD是局部依赖, 所以 R 不是2NF。此时 R 会出现冗余和异常。
- ◆ 例如, 某个医生为 N 个病人看病, 则在关系中会出现 N 个元组, 而医生的职称级别和工资就会重复 N 次。
- ◆ 如果将 R 分解为 R_1 ($Dname$, $Dlevel$, $Dsal$) 和 R_2 ($Dname$, $Pname$, $Fsum$) 后, 则局部依赖 $(Dname, Pname) \rightarrow (Dlevel, Dsal)$ 就消失了, R_1 和 R_2 都是2NF了。





■ 2NF分解算法：将关系模式R分解成2NF模式子集

- ◆ 设有关系模式 $R(U)$ ，主键是 W ， R 上还存在函数依赖 $X \rightarrow Z$ ，其中 Z 是非主属性和 $X \subset W$ ，**则 $W \rightarrow Z$ 就是一个局部依赖**。此时应该把 R 分解成两个模式：
- ◆ ① $R_1(XZ)$ ，主键是 X ；
- ◆ ② $R_2(U-Z)$ ，主键仍为 W ，外键是 X （参考 R_1 ）。
- ◆ 利用外键和主键的连接可以从 R_1 和 R_2 重新得到 R 。
- ◆ 如果 R_1 和 R_2 还不是2NF，则重复上述过程，一直到数据库模式中每一个关系模式都是2NF为止。



■ 例如

- ◆ $R(Dname, Pname, Dlevel, Dsal, Fsum)$ 中，存在FD：
 $(Dname, Pname) \rightarrow (Dlevel, Dsal)$ 和 $Dname \rightarrow (Dlevel, Dsal)$
- ◆ 则分解为： $R_1(Dname, Dlevel, Dsal)$ 和 $R_2(Dname, Pname, Fsum)$

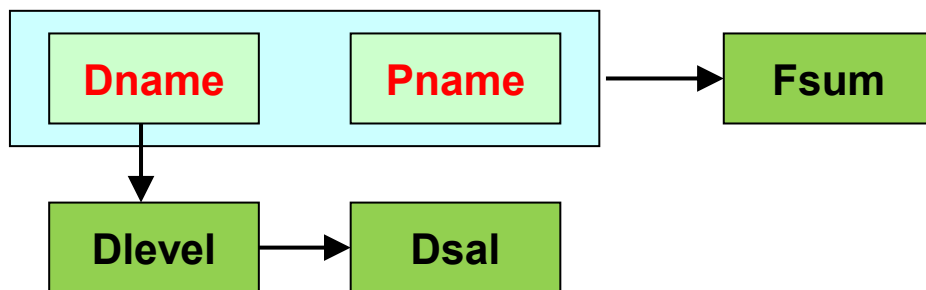


- 定义：如果关系模式 $R \in 1NF$ ，且每个**非主属性都不传递依赖**于 R 的候选码，那么称 R 属于 3NF 的模式。
- 在 3NF 中，关系模式是由主键和一组相互独立的非主属性组成的，它满足两个条件：
 - ◆ (1) R 中的非主属性相互独立；
 - ◆ (2) R 中的非主属性函数依赖于主键。



■ 3NF举例：

- ◆ R_2 (Dname, Pname, Fsum) 是2NF模式，而且也是3NF模式。
- ◆ 但是 R_1 (Dname, Dlevel, Dsal) 是2NF模式，但不一定是3NF。因为如果 R_1 中存在函数依赖 $Dname \rightarrow Dlevel$ 和 $Dlevel \rightarrow Dsal$ ，那么 $Dname \rightarrow Dsal$ 就是一个传递依赖，即 R_1 不是3NF模式。
- ◆ 此时 R_1 的关系也会出现冗余和异常。例如， R_2 中存在M个职称同为主任级别的医生，则 R_1 中需要重复存储M个相同的工资数目。
- ◆ 如果将 R_1 分解为 R_{11} (Dname, Dlevel) 和 R_{12} (Dlevel, Dsal) 后， $Dname \rightarrow Dsal$ 就不会出现在 R_{11} 和 R_{12} 中，因此 R_{11} 和 R_{12} 都是3NF的模式。





■ 3NF分解算法： 将关系模式R分解为3NF模式集。

◆ 设关系模式R (U)，主键是W，R上还存在FD $X \rightarrow Z$ ，其中Z是非主属性， $Z \not\subseteq X$ 且X不是候选键，这样 $W \rightarrow Z$ 就是一个传递依赖。此时应把R分解成两个模式：

◆ ① $R_1 (XZ)$ ，主键是X；

◆ ② $R_2 (U-Z)$ ，主键仍是W，外键是X（参考 R_1 ）。

◆ 利用外键和主键相匹配机制， R_1 和 R_2 通过连接可以重新得到 R。

◆ 如果 R_1 和 R_2 还不是3NF，则重复上述过程，一直到数据库模式中每一个关系模式都是3NF为止。

■ 例如：

◆ $R_1 (Dname, Dlevel, Dsal)$ 上存在 $Dname \rightarrow Dlevel$ 和 $Dlevel \rightarrow Dsal$ ，则 $Dname \rightarrow Dsal$ 为传递函数依赖

◆ 分解为： $R_{11} (Dname, Dlevel)$ 和 $R_{12} (Dlevel, Dsal)$



- 定理：如果 R 是3NF模式，那么 R 也是2NF模式。
- **部分依赖和传递依赖**是关系模式产生数据冗余和异常的两个重要原因。
- 由于3NF模式中不存在非主属性对候选键的部分依赖和传递依赖，因此，**3NF消除了很大一部分存储异常，具有较好的性能。**
- 而对于非1NF、1NF和2NF的关系模式，由于它们的性能较差，**通常不宜作为数据库模式**，需要将这些关系模式变换为3NF或更高级的范式。这种变换过程称为“**关系的规范化处理**”。



- 在3NF模式中，并未排除**主属性**对候选键的**传递依赖**，因此有必要提出更高一级的范式。
- BC范式（Boyce-Codd Normal Form, BCNF），由Boyce与Codd提出的，比上述的3NF又进了一步，通常认为**BCNF是修正的3NF，有时也称为扩充的3NF**。
- BCNF定义：如果关系模式 $R \in 1NF$ ，且**每个属性都不传递依赖于R的候选码**，那么称R是BCNF的模式。



■ 由BCNF的定义可以得到以下结论，一个满足BCNF的关系模式有：

- ◆ 所有**非主属性**对每一个码都是完全函数依赖。
- ◆ 所有的**主属性**对每一个不包含它的码，也是完全函数依赖。
- ◆ 没有任何属性完全函数依赖于非码的任何一组属性。



■ 举例

- ◆ 关系模式 R (Bno, Bname, Author) 的属性分别表示书号、书名和作者名。假如每个书号只有一个书名，但不同的书号可以有相同的书名；每本书可以有多个作者合写，但每个作者参与编著的书名应该互不相同。
- ◆ R 上的FD如下： $Bno \rightarrow Bname$ 和 $(Bname, Author) \rightarrow Bno$
- ◆ 因此 R 的关键码是 $(Bno, Author)$ 或 $(Bname, Author)$ ，因而模式 R 的属性都是主属性， R 是3NF模式。
- ◆ 但根据两个FD可知，属性 $Bname$ 传递依赖于关键码 $(Bname, Author)$ ，因此 R 不是BCNF。
- ◆ 例如，一本书由多个作者编写时，其书名与书号之间的联系在关系中将多次出现，会导致数据冗余和操作异常。
- ◆ 如果将 R 分解为 $R_1(Bno, Bname)$ 和 $R_2(Bno, Author)$ ，则能够解决上述问题，且 R_1 和 R_2 都是BCNF。
- ◆ 但这样分解可能会导致新的问题，例如，这个分解把 $(Bname, Author) \rightarrow Bno$ 丢失了，数据语义将会引起新的矛盾。



■ BCNF分解算法：将 R 无损分解且保持依赖地分解成3NF模式集。

- ◆① 对于关系模式 R 和 R 上成立的FD集 F ，先求出 F 的**最小依赖集**，然后再把最小依赖集中那些左部相同的FD用**合并性规则**合并起来。
- ◆② 对最小依赖集中每个FD $X \rightarrow Y$ 去构成一个模式 (XY) 。
- ◆③ 在构成的模式集中，如果每个模式都不包含 R 的候选码，那么把候选码作为一个模式放入模式集中。

■ 举例：

- ◆设关系模式 $R(ABCDE)$ ， R 的最小依赖集为 $\{A \rightarrow B, C \rightarrow D\}$ 。从依赖集可知 R 的**候选码为ACE**。
- ◆先根据最小依赖集，可知 $\rho = \{AB, CD\}$ 。然后再加入由候选码组成的模式 ACE 。因此最后结果 $\rho = \{AB, CD, ACE\}$ 是一个3NF模式集， R 相对于该依赖集是无损分解且保持函数依赖。



- 定理：如果 R 是BCNF模式，那么 R 也是3NF模式。
- 但是，若 $R \in 3NF$ ，则 R 未必属于BCNF。
- 3NF和BCNF是在函数依赖的条件下对模式分解所能达到的分离程度的测度。
- 一个数据库模式中的关系模式如果都属于BCNF，那么在函数依赖范畴内，它已实现了彻底的分离，已消除了插入和删除的异常。
- 3NF的“不彻底性”表现在可能存在主属性对码的部分依赖和传递依赖。



- 满足范式要求的数据库设计是结构清晰的，同时可避免**数据冗余和操作异常**。这并意味着**不符合范式要求的设计一定是错误的**。
- 关系模式分解一般应具有3个特性：
 - ◆ 达到BCNF，或3NF；
 - ◆ 无损分解；
 - ◆ 保持函数依赖。
- 数据库设计者在设计和关系数据库时，应做权衡，**尽可能**使数据库模式保持最好的特性。
 - ◆ 一般尽可能设计成**BCNF模式集**。
 - ◆ 如果设计成BCNF模式时达不到保持函数依赖的特点，那么只能降低要求，设计成**3NF模式集**，以求达到**保持函数依赖和无损分解**的特点。

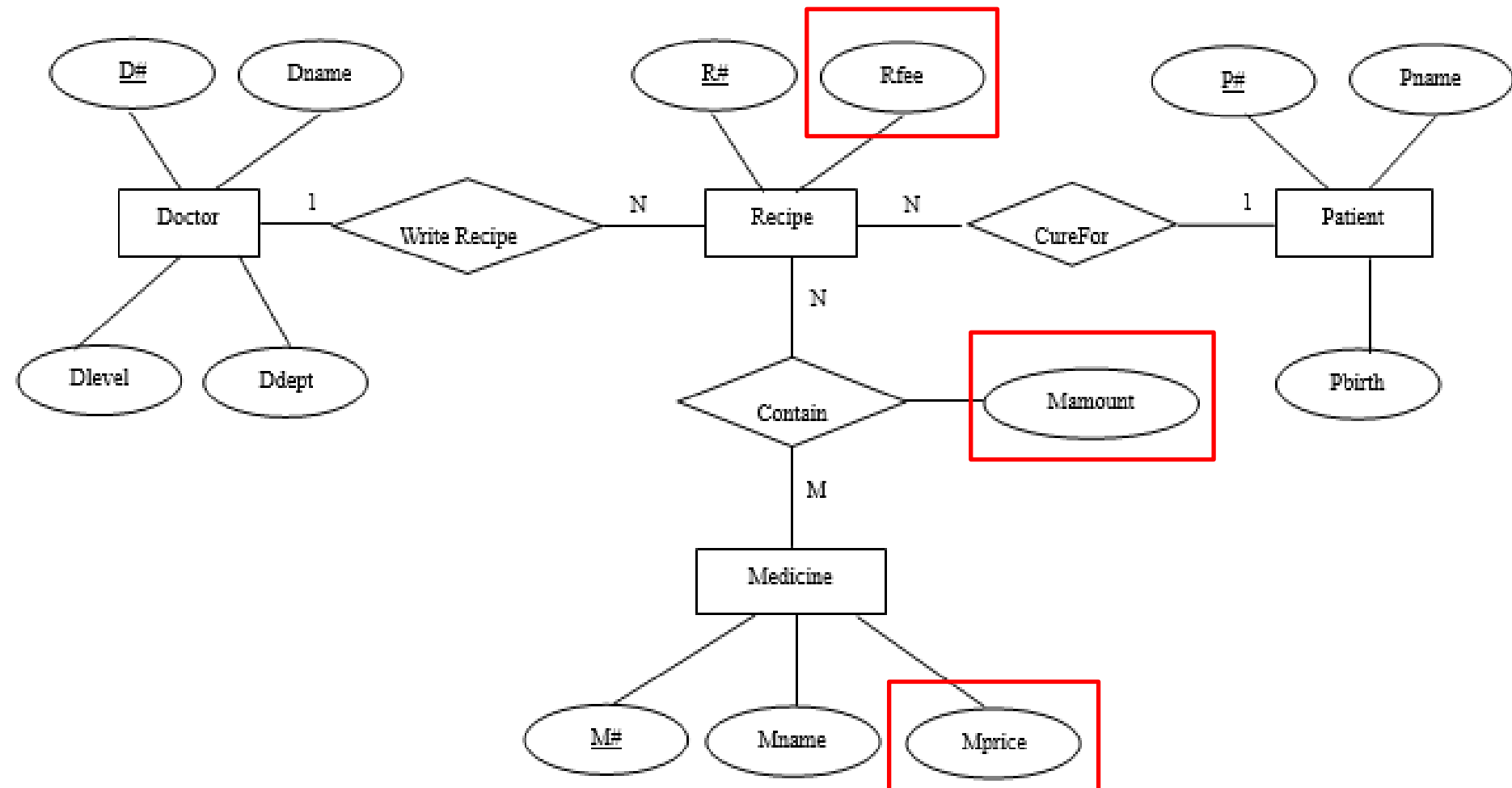


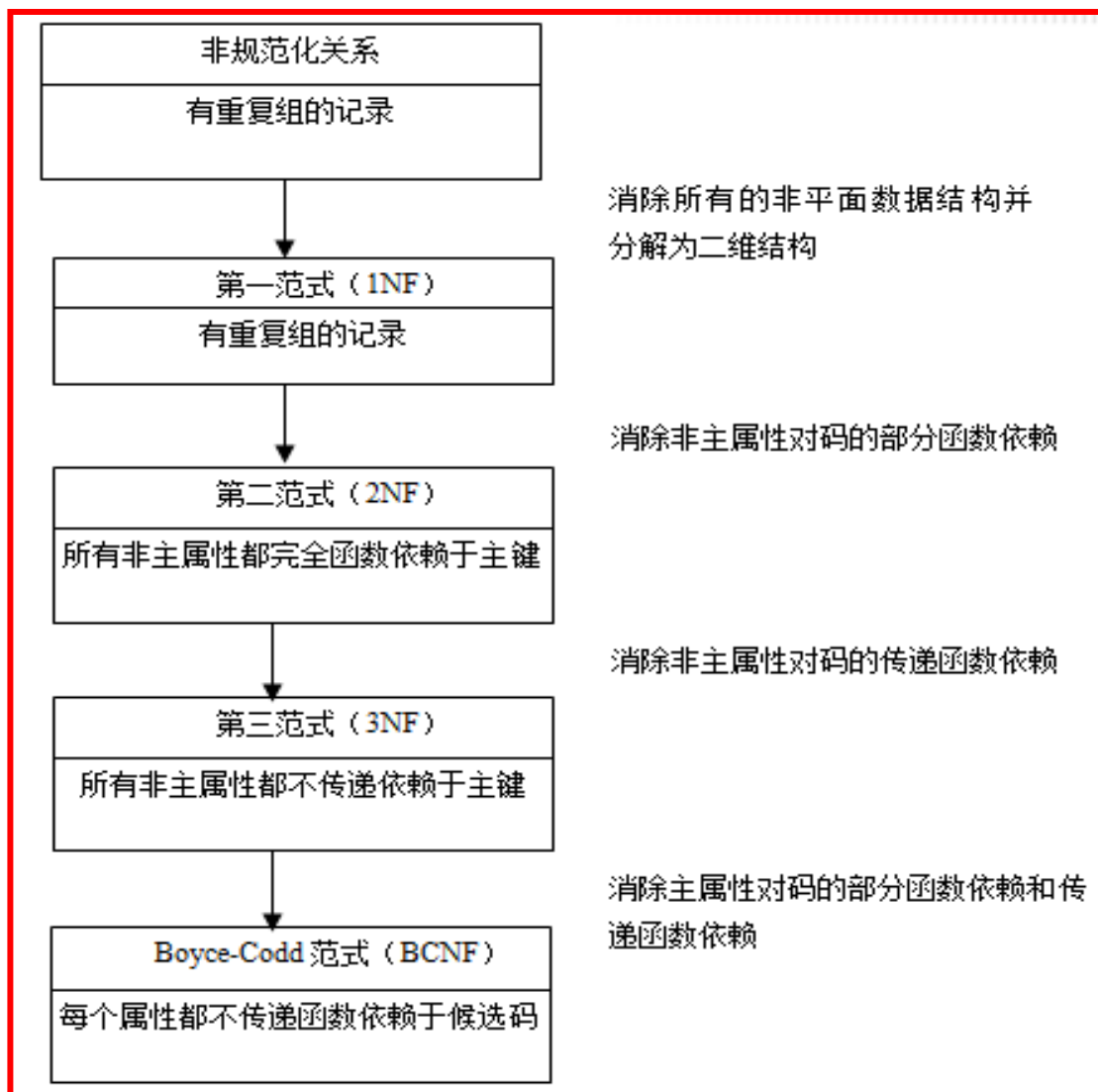
■ 一个好的模式设计方法应符合3条原则：**表达性、分离性和最小冗余性**。

- ◆ **表达性**涉及两个数据库模式的等价问题，即**数据等价和语义等价**，分别用**无损分解和保持依赖集**来衡量。
- ◆ **分离性**是指在关系中只存储有**直接联系**的属性值，而不要把有间接联系的属性值放在不同的表中。实际上“**分离**”就是**清除冗余和异常现象**。分离的基准是一系列范式。在分解成BCNF模式集时，分离与依赖等价有时是不兼容的。
- ◆ **最小冗余性**要求**分解后的模式个数和模式中属性总数应最少**。目的是节省存储空间，提高操作效率，消除不必要的冗余。但要注意，实际使用时并不一定要达到最小冗余。因为**有时带点冗余对提高查询速度是有好处的**。尤其对于那些更新频度不高，查询频度极高的数据库系统更是如此。



■ ? 思考：请分析下图中**Rfee**的是否合理？







本章内容



1

关系模式设计问题

2

函数依赖

3

模式分解

4

规范化



5

*多值依赖

6

*连接依赖



■ 多值依赖例子

- ◆ 就诊关系模式 R (Dno , Pno , $Ptel$) 的属性分别表示医生编号、患者编号、患者电话号码。该模式描述了患者就诊的医生和患者的电话号码两种独立的信息。
- ◆ 在模式 R 中, 不存在非平凡的函数依赖, 关键码由3个属性组成。模式 R 已经是BCNF, 因此模式 R 不能根据函数依赖得到进一步的分解。
- ◆ 但是模式 R 的关系中存在着**数据冗余**。例如, 患者 p_1 看过两位医生 d_1 和 d_2 , 而患者 p_1 具有两个不同的电话号码 t_1 和 t_2 , 因此需要在表中存储4个元组, 即每位患者去看一次医生, 系统就需要存储多个元组 (元组个数为该患者的电话号码个数)

就诊模式 R 的多值实例

Dno	Pno	$Ptel$
d_1	p_1	t_1
d_1	p_1	t_2
d_2	p_1	t_1
d_2	p_1	t_2
d_3	p_2	t_1
d_3	p_2	t_2
d_3	p_2	t_3



■ 多值依赖例子

- ◆ 就诊关系模式 $R(Dno, Pno, Ptel)$ 的属性分别表示医生编号、患者编号、患者电话号码。该模式描述了患者就诊的医生和患者的电话号码两种独立的信息。
- ◆ 如果将模式 R 分解为: $R_1(Dno, Pno)$ 和 $R_2(Pno, Ptel)$, 则上述冗余就可以得到消除。
- ◆ 产生这个问题的原因: **Dno与Ptel之间的联系不是直接的联系, 而是间接的联系**。把有**间接联系**的属性放在一个模式中就会产生**数据冗余和异常现象**。
- ◆ 在模式 R 中, 一位医生可以为多位患者看病 (一对多的联系), 一位患者可以有多个电话号码 (一对多的联系), 但是医生与患者电话之间没有直接的联系。
- ◆ 这种**属性间的一对多联系**称为**多值依赖**。



- 定义：设 U 是关系模式 R 的属性集， X 和 Y 是 U 的子集， $Z=U-X-Y$ ， xyz 表示属性集 XYZ 的值。对于 R 的关系 r ，在 r 中存在元组 (x, y_1, z_1) 和 (x, y_2, z_2) 时，也存在元组 (x, y_2, z_1) 和 (x, y_1, z_2) ，那么称**多值依赖**（Multivalued Dependency，简记MVD） $X \twoheadrightarrow Y$ 在模式 R 上成立。
- 例如：模式 $R(Dno, Pno, Ptel)$ 的属性值之间的一对多的联系可以用下列MVD表示：
 - ◆ $Pno \twoheadrightarrow Dno$
 - ◆ $Pno \twoheadrightarrow Ptel$



- 定义：设 D 是关系模式 R 上成立的 FD 和 MVD 集合。如果 D 中的每个非平凡的 MVD $X \twoheadrightarrow Y$ 的左部 X 都是 R 的超码，那么 R 是 4NF 的模式。
- 举例：在就诊模式 $R(Dno, Pno, Ptel)$ 中，码是 $(Dno, Pno, Ptel)$ ，在 MVD $Pno \twoheadrightarrow Dno$ 和 $Pno \twoheadrightarrow Ptel$ 的左部都未包含码，因此 R 不是 4NF 的模式。若将 R 分解为 $R_1(Dno, Pno)$ 和 $R_2(Pno, Ptel)$ ，则 R_1 和 R_2 都是 4NF。
- 关系模式 R 是 4NF 的模式，则 R 一定是 BCNF 的模式。4NF 是 BCNF 的直接推广。



本章内容



1

关系模式设计问题

2

函数依赖

3

模式分解

4

规范化

5

*多值依赖



6

*连接依赖



- 函数多值依赖定义为一个模式无损分解为两个模式，类似地，对于一个模式无损分解成 n 个模式的数据依赖，称为连接依赖。

- 定义：

设 U 是关系模式 R 的属性集， $R_1, R_2, \dots, R_n$ 是 U 的子集，并且满足 $U = R_1 \cup R_2 \cup \dots \cup R_n$ ， $\rho = \{R_1, R_2, \dots, R_n\}$ 是 R 的一个分解。如果对于 R 的每个关系 r 都有 $m_\rho(r) = r$ ，那么称连接依赖（Join Dependency, JD）在模式 R 上成立，记为 $*(R_1, R_2, \dots, R_n)$ 。

- 举例：设关系模式 R (DPM) 的属性分别表示医生、患者和药品，表示医生给患者开具的处方关系。如果规定模式 R 的关系是三个二元投影 (DP, PM, MD) 的连接，而不是其中任何两个的连接。那么模式 R 中存在一个连接依赖。



- 5NF定义：如果关系模式 R 的每个JD均由 R 的候选码蕴涵，那么称 R 是5NF的模式，有时也称投影连接范式（Project-Join NF, PJNF）。
- 连接依赖也是现实世界属性间联系的一种抽象，是语义的体现。但是它并不像FD和MVD的语义那么直观，要判断一个模式是否为5NF比较困难。
- 对于连接依赖，已经找到一些推理规则，但尚未找到完备的推理规则集。
- 可以证明，5NF的模式也一定是4NF的模式。



本章小结



- 不好的关系模式存在的问题
- 函数依赖、部分函数依赖、完全函数依赖、传递函数依赖
- 逻辑蕴涵、属性集闭包、最小函数依赖集
- Armstrong公理
- 模式分解：无损分解、函数依赖保持
- 范式：1NF、2NF、3NF、BCNF
- 模式设计原则



思考?



- 设关系模式 $R(A, B, C, D, E)$ ， R 上的函数依赖集 $F = \{A \rightarrow B, C \rightarrow D, D \rightarrow E\}$ ， R 的候选键是（ ）。
- 如果关系模式 $R \in 1NF$ ，且每个非主属性都完全函数依赖于 R 的候选码，则称 R 属于（ ）。
- 如果关系模式 R 上有函数依赖 $AB \rightarrow C$ 和 $A \rightarrow C$ ，则 R 中存在（ ）依赖。
- 在关系模式 $R(U)$ 中，如果 $X \rightarrow Y$ ， $Y \rightarrow Z$ ，且 Y 不是 X 的子集，不存在 $X \leftarrow \rightarrow Y$ 的情况，则称 Z _____依赖于 X 。
- 已知关系 $R(A, B, C, D, E)$ 及其上的函数相关性集合， $F = \{A \rightarrow B, E \rightarrow A, CE \rightarrow D\}$ ，该关系的候选关键字是（ ）。